Unidad 3. Patrones de comportamiento: Strategy y Observer

Patrones de Diseño · Facultad de Ingeniería · Notas de clase para uso académico

1. Strategy

Strategy encapsula una familia de algoritmos en clases separadas con una interfaz común y permite cambiarlos sin modificar a quien los usa (el contexto). Sustituye cadenas de condicionales del tipo "si el método es A haz esto, si es B haz aquello".

```
interface CalculoRiesgo { double puntaje(Estudiante e); }

class SoloNota implements CalculoRiesgo {
    public double puntaje(Estudiante e) { return 5 - e.promedio(); }
}
class NotaYTendencia implements CalculoRiesgo {
    public double puntaje(Estudiante e) { return (5 - e.promedio()) - e.tendencia(); }
}

class Alertas {
    private final CalculoRiesgo estrategia;
    Alertas(CalculoRiesgo estrategia) { this.estrategia = estrategia; }
}
```

Cada estrategia se prueba por separado y agregar una nueva no modifica las existentes (principio abierto/cerrado). El costo es que el cliente debe conocer las estrategias para elegir una.

2. Observer

Observer define una relación uno a muchos: cuando el sujeto cambia de estado, notifica a todos sus observadores suscritos. El sujeto no conoce las clases concretas de los observadores, solo la interfaz, de modo que se pueden agregar reacciones nuevas sin tocarlo.

```
interface ObservadorNota { void notaRegistrada(Nota nota); }

class RegistroNotas {
    private final List<ObservadorNota> observadores = new ArrayList<>();
    void suscribir(ObservadorNota o) { observadores.add(o); }
    void registrar(Nota nota) {
        guardar(nota);
        observadores.forEach(o -> o.notaRegistrada(nota));
    }
}
// Observadores: recalcular nivel, sugerir taller, enviar alerta al docente
```

3. Riesgos de Observer

- El orden de notificación no debería importar; si importa, el diseño está acoplado.
- Un observador lento o que lanza excepciones afecta a todos los demás.

- Olvidar cancelar suscripciones produce fugas de memoria en aplicaciones de larga duración.
- Las cadenas de notificaciones (un observador que cambia otro sujeto) son difíciles de depurar.

4. Strategy y Observer en un mismo sistema

En una aplicación de seguimiento académico, Observer reacciona cuando se registra una nota y Strategy decide cómo se calcula el riesgo de cada estudiante. Ambos patrones reducen el acoplamiento: el registro de notas no sabe qué pasa después, y el módulo de alertas no sabe qué fórmula se usa.

5. Ejercicios de práctica

1. Implementa dos estrategias de descuento (porcentaje y valor fijo) para un carrito de compras.
2. Diseña un sujeto Curso que notifique cuando un estudiante baja de 3,0. Propón dos observadores.
3. ¿Qué diferencia hay entre Strategy y State? Da un ejemplo de cada uno.
4. Identifica qué principio SOLID se viola si el sujeto hace instanceof sobre sus observadores.

Fuentes consultadas

• Gamma, Helm, Johnson y Vlissides (1994). Design Patterns. Addison-Wesley. Strategy (p. 315) y Observer (p. 293).
• Refactoring.Guru. Strategy y Observer. refactoring.guru/es/design-patterns
• Wikipedia. Observer (patrón de diseño) (CC BY-SA 4.0). es.wikipedia.org

Texto original redactado para el curso a partir de las fuentes citadas.